

Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas

Universidad Complutense de Madrid

Óscar Martín — Curso 2026/27

Dos ejercicios más interesantes

1. La función `while` definida abajo sirve para expresar declarativamente el cambio de estado producido por un bucle imperativo:

```
while cond cuerpo estado =  
  if cond estado  
  then while cond cuerpo (cuerpo estado)  
  else estado
```

Piensa cuál es el tipo más general de `while`. Considera esta función imperativa:

```
función Ejemplo (x, y, z : Int)  
  inicio  
     $z \leftarrow x$   
    mientras  $x \leq y$  hacer  
       $z \leftarrow z \cdot x$   
       $x \leftarrow x + 1$   
    fin mientras  
    devolver (x, y, z)  
fin
```

Usando la función `while`, define una función

```
| ejemplo :: (Int, Int, Int) -> (Int, Int, Int)
```

que simule la ejecución de la función *Ejemplo*: `ejemplo (2, 3, 1)  $\rightsquigarrow$  (4, 3, 12)`. Declara y define también dos funciones `ifThenElse` e `ifThen` que expresen declarativamente los *if* imperativos. Por ejemplo:

```
ifThen (0 <= 1) (\(x, y, _) -> (x, y, 0)) (2, 3, 1)  $\rightsquigarrow$  (2, 3, 0).
```

2. En *notación polaca inversa* los operadores se escriben detrás de sus operandos: $3 + 4$ se escribe `3 4 +`; de igual forma, $(3 + 4) \cdot 2$ se escribe `3 4 + 2 *`. Con esta notación, ni los paréntesis ni las prioridades entre operadores son necesarios. Para evaluar una expresión se recorre de izquierda a derecha con ayuda de una pila: cada número se apila; cada operador desapila los dos últimos números, opera con ellos (convengamos en que el primero que sale es el operando de la derecha) y apila el resultado. Al terminar, la pila debe contener exactamente un número, que es el valor de la expresión.

Declara y define una función `evalua`. Debe recibir una cadena de caracteres que contenga una expresión en notación polaca inversa con los números y los operadores separados por espacios. Hay varias formas en que una expresión puede estar mal formada: sobran o faltan operandos, se divide entre cero, se eleva a un exponente negativo o contiene caracteres que no son números ni operadores. Para empezar, puedes suponer que la cadena de entrada contiene una expresión bien formada; después, puedes ir introduciendo tratamiento de errores. Por ejemplo, tu función puede devolver un valor booleano que diga si ha habido error, acompañando al resultado si todo ha ido bien. Como pila puedes usar una lista. Te pueden ser útiles las funciones predefinidas `words`, que separa una cadena en una lista de *palabras*, y `read`, que convierte una cadena en un número. Ejemplo:

- `evalua "3 2 ^ 11 2 7 - + *"` $\rightsquigarrow$ `54`

O, si tienes tratamiento de errores:

- `evalua "3 2 ^ 11 2 7 - + *"` $\rightsquigarrow$ `(True, 54)`
- `evalua "5 +"` $\rightsquigarrow$ `(False, 0)`
- `evalua "1 2 3 +"` $\rightsquigarrow$ `(False, 0)`